

DOSSIER DE PROJET

Gestion sécurisée des dossiers administratifs

Application web pour collectivités territoriales

Présenté par :

SecureApp

Entreprise spécialisée en développement d'applications sécurisées

Référence du projet : PRJ-2025-04

Version : 1.0

Date : Avril 2025

DOCUMENT CONFIDENTIEL

Ce document contient des informations confidentielles et ne doit pas être reproduit sans l'autorisation de SecureApp.

1. Présentation de l'entreprise : SecureApp	1
1.1 Valeurs et engagements	1
1.2 Domaines d'activité principaux	1
1.3 Organigramme	2
2. Présentation du Service Conception et Développement	2
2.1 Méthodologie de travail	3
2.2 Missions principales du service	3
3. Compétences professionnelles mobilisées sur le projet	3
3.1 Analyse des besoins et rédaction du cahier des charges	3
3.2 Conception d'architectures logicielles	4
3.3 Développement d'applications sécurisées	4
3.4 Tests et validation	4
3.5 Déploiement et maintenance	4
3.6 Documentation et communication	4
3.7 Veille technologique et sécuritaire	4
3.8 Travail en équipe et gestion de projet agile	5
4. Cahier des charges / Expression des besoins	5
4.1 Contexte et objectifs	5
4.2 Utilisateurs cibles	5
4.3 Fonctionnalités attendues	5
4.4 Contraintes à respecter	6
4.5 Livrables attendus	6
4.6 Environnement technique	6
5. Gestion de projet	7
5.1 Méthodologie	7
5.2 Phases du projet et planning prévisionnel	7
5.3 Suivi du projet	8
5.4 Environnement humain (acteurs et rôles)	8
5.5 Objectifs qualité	8
6. Spécifications fonctionnelles	9
6.1 Contraintes à respecter	9
6.2 Livrables attendus	9
6.3. Architecture logicielle (multicouche)	10
a. Schéma général	10
b. Explication des couches	10
6.4 Maquettes des écrans clés et parcours de navigation	10
6.5 MCD (Modèle Conceptuel de Données)	10
6.6 MLD (Modèle Logique de Données)	12
6.7 MPD (Modèle physique de Données)	13
1. Table Utilisateur	13
2. Table Dossier	13

3. Table Historique	14
4. Table Pièce Jointes	14
7. Script SQL de création de la BDD / Tables	14
8. Diagrammes UML	16
8.1 Diagramme de cas d'utilisation	16
8.2 Diagramme de séquence :	17
8.3 Diagramme de classes	17
9. Maquettes filaires des écrans clés et parcours de navigation	18
9.1 Page de connexion	18
9.2 Tableau de bord (Accueil)	19
9.3 Formulaire de création/modification de dossier	20
10. Spécifications techniques détaillées	20
10.1 Sécurité des données	20
10.2 Authentification et gestion des sessions	21
10.3 Gestion des autorisations	21
10.4 Gestion des exceptions et des erreurs	21
10.5 Conformité ANSSI et RGPD	22
10.6 Accessibilité (RGAA)	22
10.7 Performance et éco-conception	22
10.8 Déploiement et intégration continue	22
11. Points clés de la conformité sécuritaire	23
12. Plan de test	23
12.1 Fonctionnalité "Création de dossier"	23
12.2 Jeu d'essai "Création de dossiers"	24
13. Extraits de code	24
13.1 Entité Dossier	24
13.2 Entité Utilisateur	25
13.3 Repository JPA	25
13.4 Service métier (exemple création dossier)	25
13.5 Contrôleur REST sécurisé	26
13.6 Sécurité (Spring Security basique)	27
13.7 Configuration MySQL	27
14. Programmation Orientée Objet : Facilitation de la maintenance et de l'évolution	28
15. Éléments de sécurité mis en œuvre dans l'application	28
15.1 Sécurisation des données	29
15.2 Authentification et contrôle d'accès	29
15.3 Journalisation et traçabilité	29
15.4 Protection contre les attaques courantes	29
15.5 Conformité réglementaire	30

1. Présentation de l'entreprise : SecureApp

SecureApp est une entreprise française fondée en 2018, spécialisée dans le développement d'applications web sécurisées. Elle compte aujourd'hui 80 salariés répartis sur deux sites : Bordeaux (siège social) et Nantes.

1.1 Valeurs et engagements

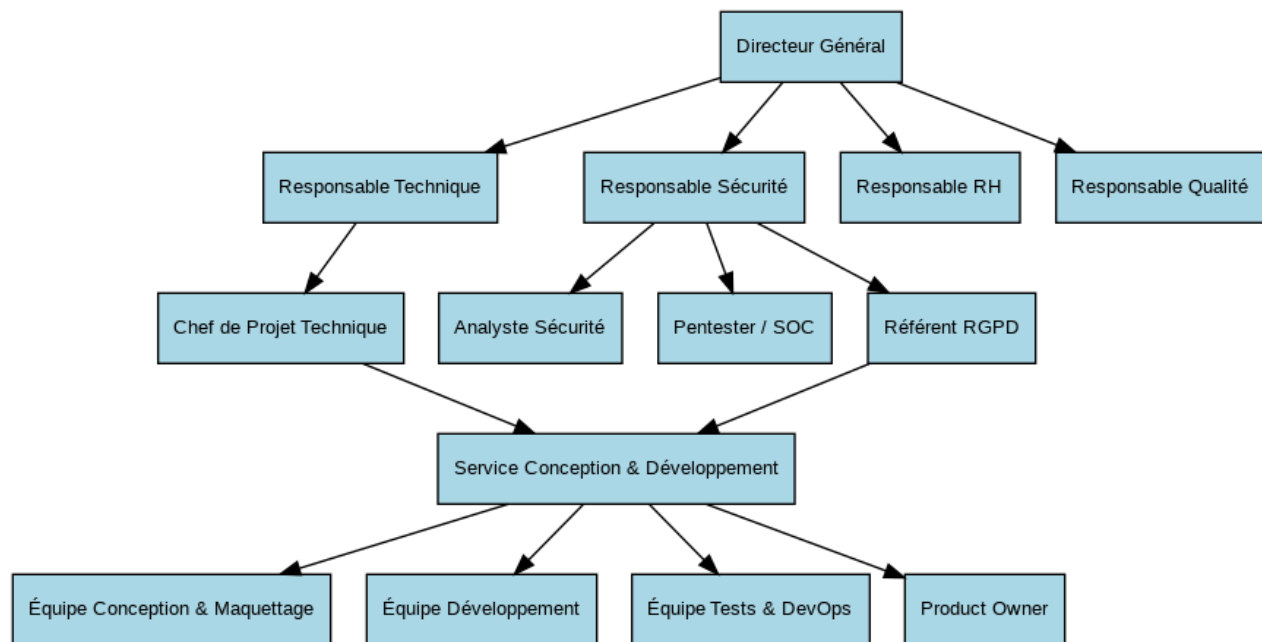
- **Respect du RGPD** (Règlement Général sur la Protection des Données) et des recommandations de l'ANSSI (Agence nationale de la sécurité des systèmes d'information).
- **Accessibilité** : conformité au RGAA (Référentiel Général d'Amélioration de l'Accessibilité).
- **Éco-conception** : développement de solutions numériques à faible impact environnemental.

1.2 Domaines d'activité principaux

Domaine	Description
Finance & Assurance	Plateformes sécurisées pour la gestion des transactions et des risques financiers
Secteur public	Systèmes numériques pour la gestion sécurisée des données citoyennes

Tableau 1 : Domaines d'activité principaux de SecureApp

1.3 Organigramme



2. Présentation du Service Conception et Développement

Le **Service Conception et Développement** de SecureApp est composé de 20 collaborateurs organisés en trois équipes complémentaires :

Équipe	Effectif	Rôle principal
Conception & Maquettage	5	Analyse des besoins, réalisation de maquettes, conception d'architectures multicouches sécurisées
Développement	10	Développement des interfaces utilisateurs, composants métier, accès aux données (SQL/NoSQL)
Tests & DevOps	5	Élaboration des plans de tests, déploiement, intégration continue, CI/CD, gestion DevOps

Tableau 2 : Equipes du service Conception et Développement

2.1 Méthodologie de travail

- **Méthode agile** : cycles courts, sprints, retours utilisateurs rapides.
- **Utilisation généralisée de conteneurs** (Docker, Kubernetes) et d'outils collaboratifs (Git, outils de ticketing, CI/CD).
- **Collaboration étroite** entre les équipes pour garantir la cohérence du développement à la production.

2.2 Missions principales du service

- Analyse et formalisation des besoins clients.
- Conception d'architectures logicielles robustes et sécurisées.
- Développement, tests, déploiement et maintenance des applications.
- Veille technologique et sécuritaire continue.

3. Compétences professionnelles mobilisées sur le projet

Le projet de gestion sécurisée des dossiers administratifs mobilise l'ensemble des compétences clés du référentiel CDA. Les principales compétences requises et leur mise en œuvre dans le contexte du projet :

3.1 Analyse des besoins et rédaction du cahier des charges

- Recueillir et formaliser les besoins du client.
- Rédiger un cahier des charges précis intégrant les contraintes réglementaires (RGPD, RGAA, ANSSI).

3.2 Conception d'architectures logicielles

- Concevoir une architecture multicouche (présentation, métier, données).
- Modéliser les données (MCD, MPD) et les flux applicatifs (UML).

3.3 Développement d'applications sécurisées

- Développer des interfaces utilisateurs accessibles et ergonomiques.
- Implémenter les règles métiers et la logique applicative.

- Gérer l'accès sécurisé aux données (SQL/NoSQL).
- Intégrer des mécanismes d'authentification, d'autorisation et de gestion des exceptions.

3.4 Tests et validation

- Élaborer et exécuter des plans de tests fonctionnels et techniques.
- Automatiser les tests et intégrer la validation continue (CI/CD).

3.5 Déploiement et maintenance

- Utiliser des outils de conteneurisation (Docker, Kubernetes) pour le déploiement.
- Assurer la maintenance corrective et évolutive de l'application.

3.6 Documentation et communication

- Rédiger des documentations techniques et fonctionnelles complètes.
- Présenter les solutions et les choix techniques aux parties prenantes.

3.7 Veille technologique et sécuritaire

- Assurer une veille sur les vulnérabilités et les bonnes pratiques de sécurité.
- Mettre à jour les composants et les pratiques en fonction des dernières recommandations (ANSSI, RGPD...).

3.8 Travail en équipe et gestion de projet agile

- Collaborer efficacement avec les différents membres de l'équipe (conception, développement, tests, sécurité, chef de projet).
- Participer aux cérémonies agiles (sprints, revues, rétrospectives).

4. Cahier des charges / Expression des besoins

4.1 Contexte et objectifs

La collectivité territoriale souhaite disposer d'une application web sécurisée pour la gestion de ses dossiers administratifs. Cette application doit permettre aux agents habilités de créer,

consulter, modifier et supprimer des dossiers, tout en respectant les exigences réglementaires et techniques en vigueur.

4.2 Utilisateurs cibles

- **Agents administratifs** : gestion quotidienne des dossiers.
- **Responsables de service** : supervision, validation, statistiques.
- **Administrateurs système** : gestion des droits, maintenance.

4.3 Fonctionnalités attendues

- **Créer un dossier** : saisie des informations, validation des champs, enregistrement sécurisé.
- **Consulter un dossier** : recherche, affichage détaillé, respect des droits d'accès.
- **Modifier un dossier** : édition des informations selon les droits, traçabilité des modifications.
- **Supprimer un dossier** : suppression logique (archivage), gestion des droits.
- **Gestion des utilisateurs et des droits** : authentification forte, rôles différenciés (agent, responsable, admin).
- **Historique et traçabilité** : journalisation des actions (création, modification, suppression).
- **Accessibilité** : conformité RGAA pour tous les écrans et interactions.
- **Recherche et filtrage** : recherche multicritère, tri des dossiers.
- **Statistiques** : tableaux de bord pour les responsables.

4.4 Contraintes à respecter

- **Sécurité** : conformité ANSSI, chiffrement des données sensibles, gestion des sessions, protection contre les attaques courantes (XSS, CSRF, injection SQL, etc.).
- **Confidentialité** : respect du RGPD (droit à l'oubli, consentement, minimisation des données).
- **Accessibilité** : conformité RGAA (niveau AA minimum).
- **Performance** : temps de réponse rapide, optimisation des requêtes et du front-end.

- **Éco-conception** : limitation de la consommation de ressources, sobriété des interfaces.
- **Maintenabilité** : code structuré, documentation, modularité.

4.5 Livrables attendus

- Spécifications fonctionnelles et techniques détaillées.
- Maquettes filaires ou graphiques des écrans.
- Architecture logicielle multicouche.
- Modèle conceptuel (MCD) et physique (MPD) de la base de données.
- Script SQL de création/modification de la BDD.
- Diagrammes UML (cas d'utilisation, séquence, classes).
- Plan de tests détaillé et jeu d'essai.
- Synthèse de veille sécuritaire.
- Captures d'écran et extraits de code.

4.6 Environnement technique

- **Front-end** : framework moderne compatible RGAA.
- **Back-end** : langage sécurisé : Java, API RESTful.
- **Base de données** : SQL.
- **Déploiement** : conteneurs Docker, CI/CD, hébergement sécurisé.

5. Gestion de projet

5.1 Méthodologie

Le projet est piloté en **mode agile** (type Scrum), avec des cycles courts (sprints de 2 à 3 semaines) et des retours réguliers des utilisateurs pour adapter la solution aux besoins réels.

5.2 Phases du projet et planning prévisionnel

Phase	Durée estimée	Livrables principaux
Analyse des besoins	1 semaine	Cahier des charges, expression des besoins
Conception détaillée	2 semaines	Spécifications fonctionnelles, maquettes, MCD/MPD, architecture
Développement itératif	4 semaines	Application web (front + back), scripts SQL, tests unitaires
Recette et validation	1 semaine	Plan de tests, correction des anomalies
Déploiement et formation	1 semaine	Mise en production, documentation, formation utilisateurs

Tableau 3 : Phases du projet et planning

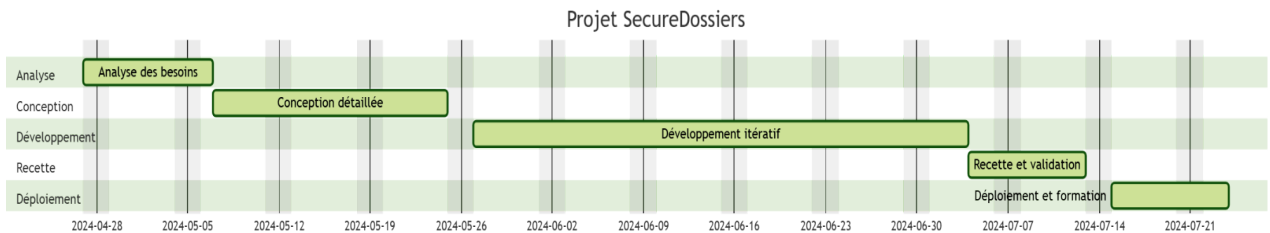


Figure 2 : Diagramme de Gantt du projet

Total prévisionnel : 9 semaines

5.3 Suivi du projet

- **Outils de gestion** : Jira (suivi des tâches et des sprints), GitHub (gestion du code et CI/CD), Slack (communication).
- **Réunions régulières** : daily meeting, revue de sprint, rétrospective.
- **Indicateurs de suivi** : avancement des tâches, couverture des tests, respect des délais, gestion des risques.

5.4 Environnement humain (acteurs et rôles)

Acteur	Rôle
Chef de projet	Pilote le projet, organise les sprints, valide les livrables
Concepteur-développeur	Analyse, conçoit, développe, documente
Développeur front-end	Réalise les interfaces utilisateur accessibles et ergonomiques
Développeur back-end	Implémente la logique métier, la sécurité, l'accès aux données
Responsable sécurité	Valide la conformité ANSSI, RGPD, supervise la veille sécuritaire
Testeur	Rédige et exécute les plans de tests, remonte les anomalies
Utilisateur référent	Fournit les retours, valide l'adéquation fonctionnelle

Tableau 4 : Acteurs et rôles

5.5 Objectifs qualité

- **Sécurité** : conformité ANSSI, RGPD, gestion des vulnérabilités.
- **Accessibilité** : conformité RGAA (niveau AA minimum).
- **Performance** : temps de réponse < 2s pour les principales opérations.
- **Maintenabilité** : code documenté, architecture modulaire, tests automatisés.
- **Satisfaction utilisateur** : application intuitive, adaptée aux besoins métiers.

6. Spécifications fonctionnelles

6.1 Contraintes à respecter

- **Sécurité** : conformité ANSSI, chiffrement des données sensibles, gestion des droits d'accès, journalisation des actions, protection contre les attaques courantes (XSS, CSRF, injection SQL...).
- **Confidentialité** : respect du RGPD (droit à l'oubli, consentement, minimisation des données personnelles).

- **Accessibilité** : conformité RGAA (niveau AA minimum) pour tous les écrans et interactions.
- **Performance** : temps de réponse < 2 secondes pour toutes les opérations principales.
- **Éco-conception** : interfaces sobres, limitation du nombre de requêtes et de la consommation de ressources.
- **Maintenabilité** : code structuré, modulaire, documenté, tests automatisés.

6.2 Livrables attendus

- **Documentation des spécifications fonctionnelles et techniques**
- **Maquettes filaires ou graphiques** des écrans principaux
- **Architecture logicielle** détaillée (schéma multicouche)
- **Modèle conceptuel (MCD)** et **modèle physique (MPD)** de la base de données
- **Script SQL** de création/modification de la BDD
- **Diagrammes UML** (cas d'utilisation, séquence, classes)
- **Plan de tests** détaillé et jeu d'essai
- **Synthèse de veille sécuritaire**
- **Captures d'écran** et **extraits de code** (IU, métier, accès données)

6.3. Architecture logicielle (multicouche)

a. Schéma général

- **Couche présentation (front-end)** : interfaces utilisateurs accessibles, communication avec l'API via HTTPS, gestion des formulaires et des retours d'erreur.
- **Couche métier (back-end)** : logique applicative, gestion des droits, validation des données, journalisation des actions.
- **Couche données** : base de données sécurisée (SQL), gestion des accès, sauvegardes, chiffrement des données sensibles.

b. Explication des couches

- **Présentation** : conforme RGAA, gère l'affichage et la saisie des données.

- **Métier** : Java/Spring gère la logique, la sécurité, l'authentification.
- **Données** : MySQL, tables/collections sécurisées, requêtes optimisées.

6.4 Maquettes des écrans clés et parcours de navigation

- **Page de connexion** (authentification forte)
- **Tableau de bord** (liste des dossiers, filtres, accès rapide)
- **Formulaire de création/modification de dossier**
- **Page de consultation d'un dossier**
- **Gestion des utilisateurs et des droits**
- **Historique des actions (audit log)**
- **Page d'administration (pour les admins)**

6.5 MCD (Modèle Conceptuel de Données)

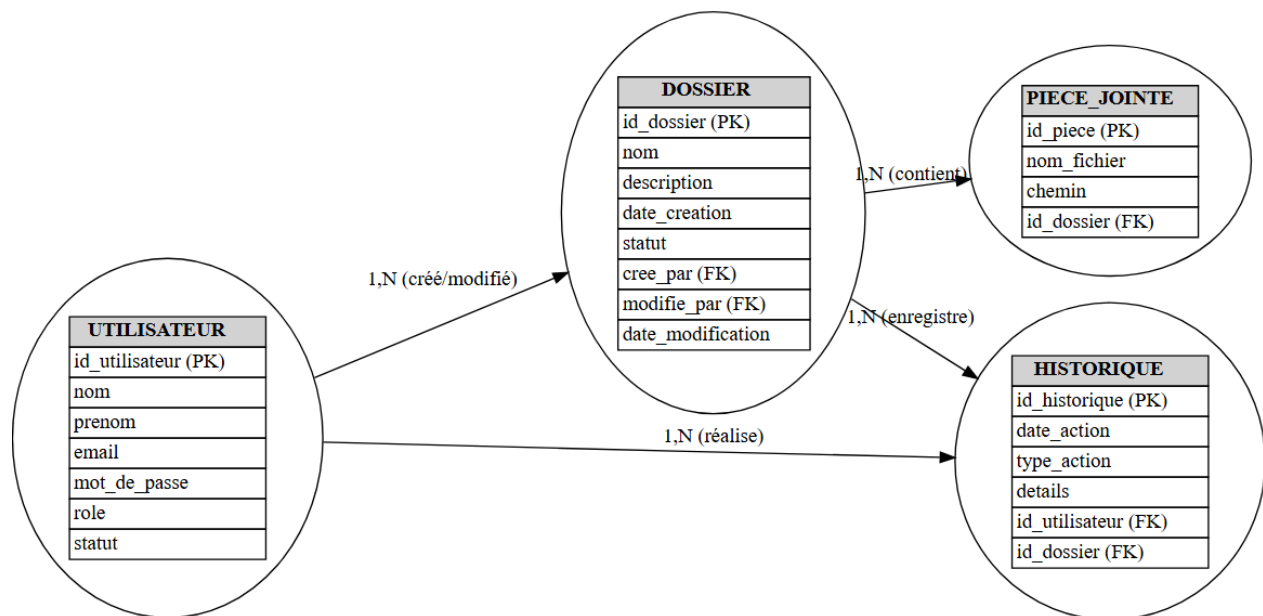


Figure 4 : MCD (Modèle Conceptuel de Données) du projet

6.6 MLD (Modèle Logique de Données)

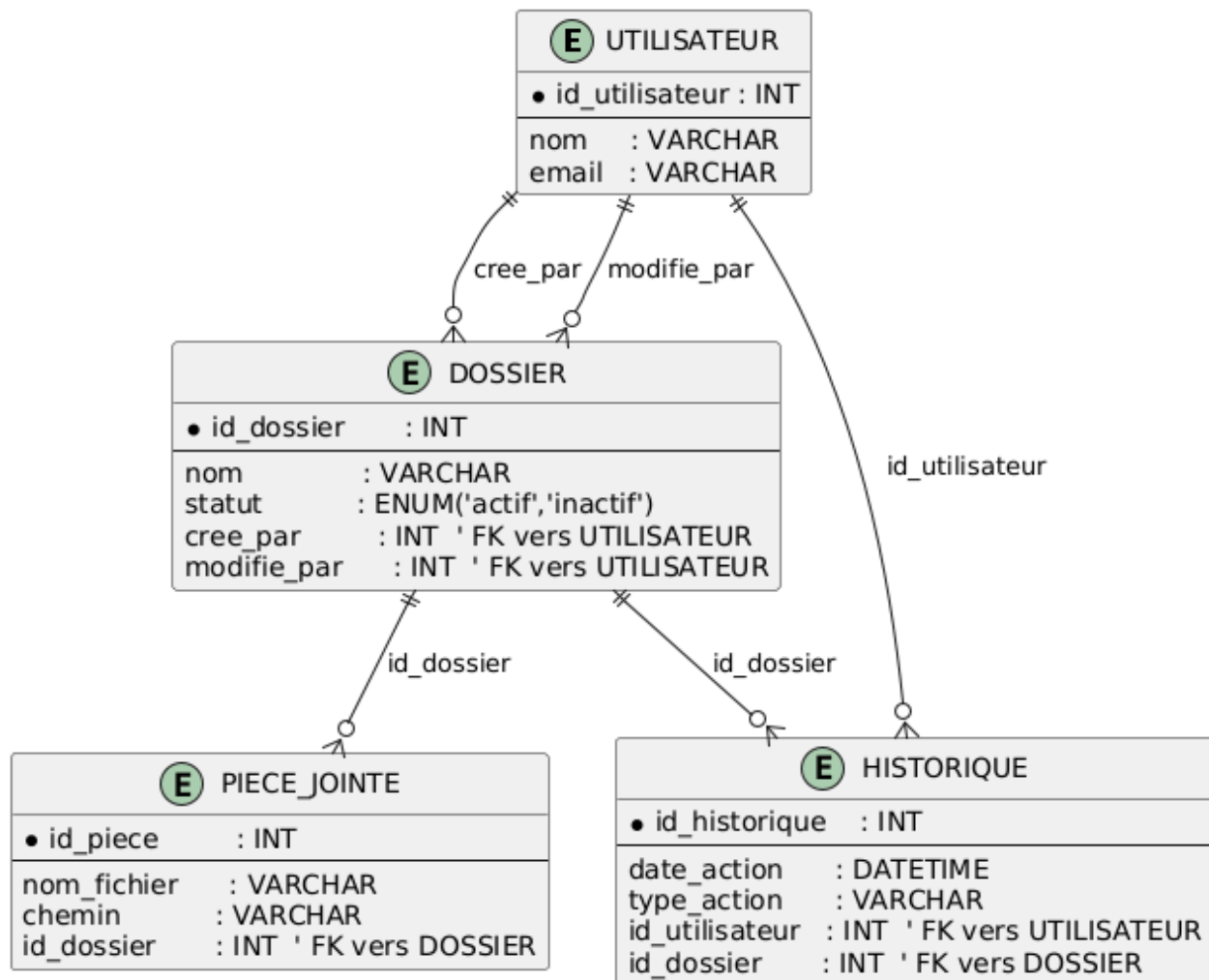


Figure 5 : MLD (Modèle Logique de Données) du projet

6.7 MPD (Modèle physique de Données)

1. Table Utilisateur

Colonne	Type	Contraintes
id_utilisateur	INT AUTO_INCREMENT	PK
nom	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE

Tableau 5 : Table SQL Utilisateur

2. Table Dossier

Colonne	Type	Contraintes
id_dossier	INT AUTO_INCREMENT	PK
nom	VARCHAR(150)	NOT NULL
statut	ENUM('actif','inactif')	NOT NULL, DEFAULT 'actif'
cree_par	INT	NOT NULL, FK → UTILISATEUR(id_utilisateur)
modifie_par	INT	NOT NULL, FK → UTILISATEUR(id_utilisateur)
date_modification	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Tableau 6 : Table SQL Dossier

3. Table Historique

Colonne	Type	Contraintes
id_historique	INT AUTO_INCREMENT	PK
date_action	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
type_action	VARCHAR(50)	NOT NULL
id_utilisateur	INT	NOT NULL, FK → UTILISATEUR(id_utilisateur)
id_dossier	INT	NOT NULL, FK → DOSSIER(id_dossier)

Tableau 7 : Table SQL Historique

4. Table Pièce Jointes

Colonne	Type	Contraintes
id_piece	INT AUTO_INCREMENT	PK
nom_fichier	VARCHAR(200)	NOT NULL
chemin	VARCHAR(255)	NOT NULL
id_dossier	INT	NOT NULL, FK → DOSSIER(id_dossier)

Tableau 8 : Table SQL Pièces jointes

7. Script SQL de création de la BDD / Tables

```
-- Création de la base de données
CREATE DATABASE gestion_dossiers;
USE gestion_dossiers;

-- Création de la table UTILISATEUR
CREATE TABLE UTILISATEUR (
    id_utilisateur INT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    mot_de_passe VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL,
    statut VARCHAR(20) NOT NULL
);

-- Création de la table DOSSIER
CREATE TABLE DOSSIER (
    id_dossier INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    description TEXT,
    date_creation TIMESTAMP NOT NULL,
    statut ENUM('actif', 'inactif') NOT NULL,
    cree_par INT NOT NULL,
    modifie_par INT,
    date_modification TIMESTAMP,
    FOREIGN KEY (cree_par) REFERENCES UTILISATEUR(id_utilisateur),
    FOREIGN KEY (modifie_par) REFERENCES UTILISATEUR(id_utilisateur)
);

-- Création de la table PIECE_JOINTE
CREATE TABLE PIECE_JOINTE (
    id_piece INT PRIMARY KEY,
    nom_fichier VARCHAR(255) NOT NULL,
    chemin VARCHAR(255) NOT NULL,
    id_dossier INT NOT NULL,
    FOREIGN KEY (id_dossier) REFERENCES DOSSIER(id_dossier)
);

-- Création de la table HISTORIQUE
CREATE TABLE HISTORIQUE (
    id_historique INT PRIMARY KEY,
    date_action TIMESTAMP NOT NULL,
    type_action VARCHAR(50) NOT NULL,
```

```
details TEXT,  
id_utilisateur INT NOT NULL,  
id_dossier INT NOT NULL,  
FOREIGN KEY (id_utilisateur) REFERENCES UTILISATEUR(id_utilisateur),  
FOREIGN KEY (id_dossier) REFERENCES DOSSIER(id_dossier)  
);
```

Figure 6 : Script SQL de création de la BDD

8. Diagrammes UML

8.1 Diagramme de cas d'utilisation

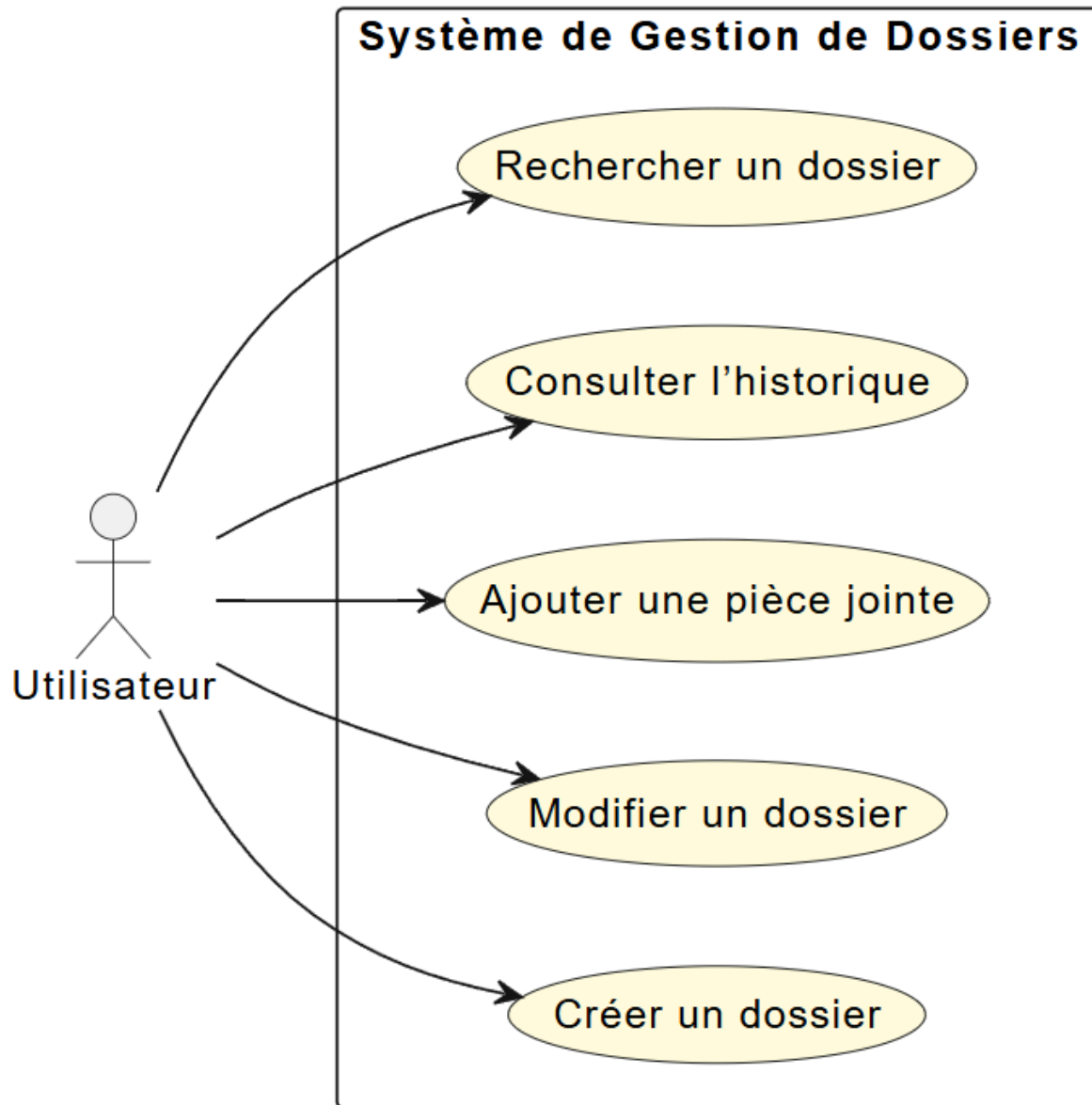


Figure 7 : Diagramme de cas d'utilisation

8.2 Diagramme de séquence :

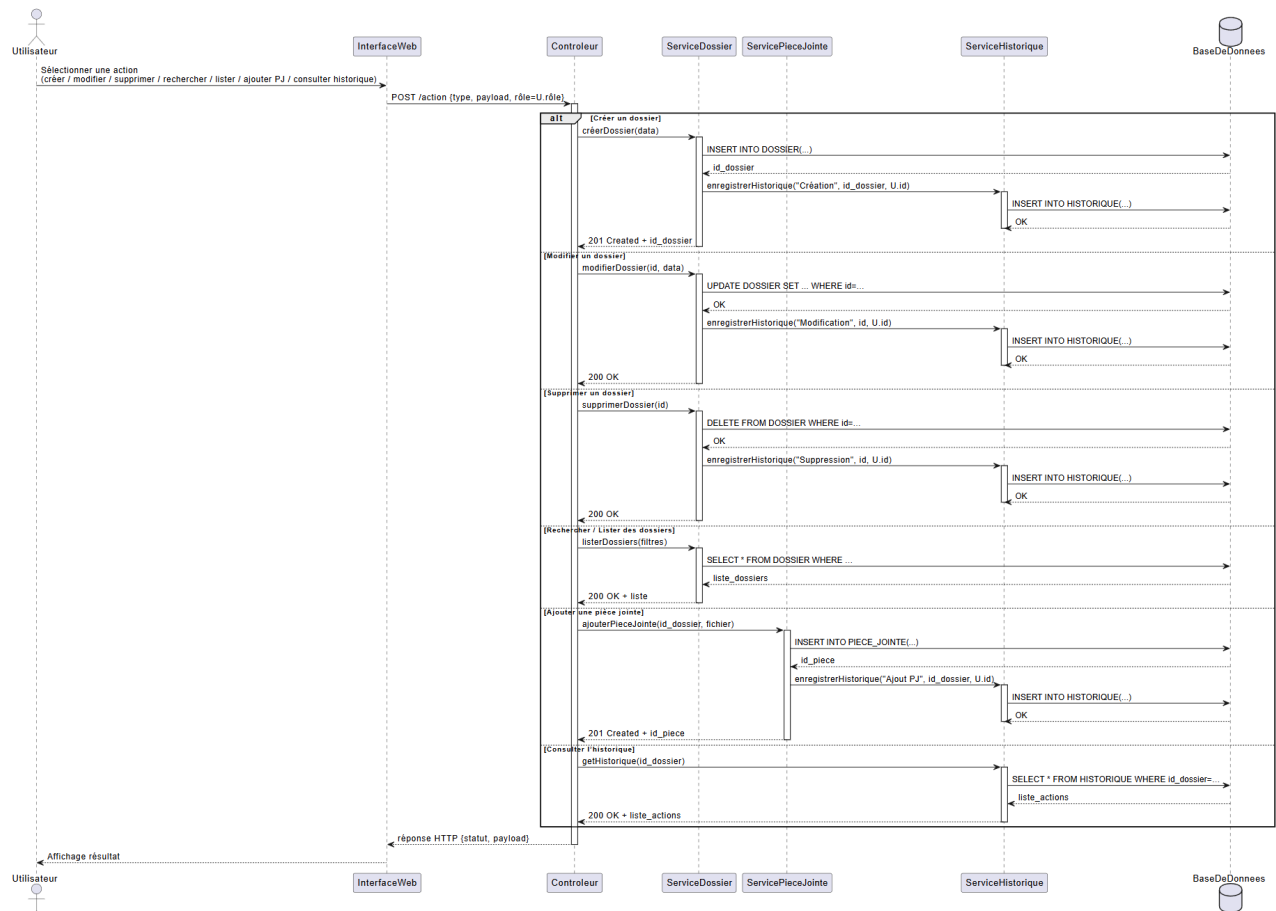


Figure 8 : Diagramme de séquences

8.3 Diagramme de classes

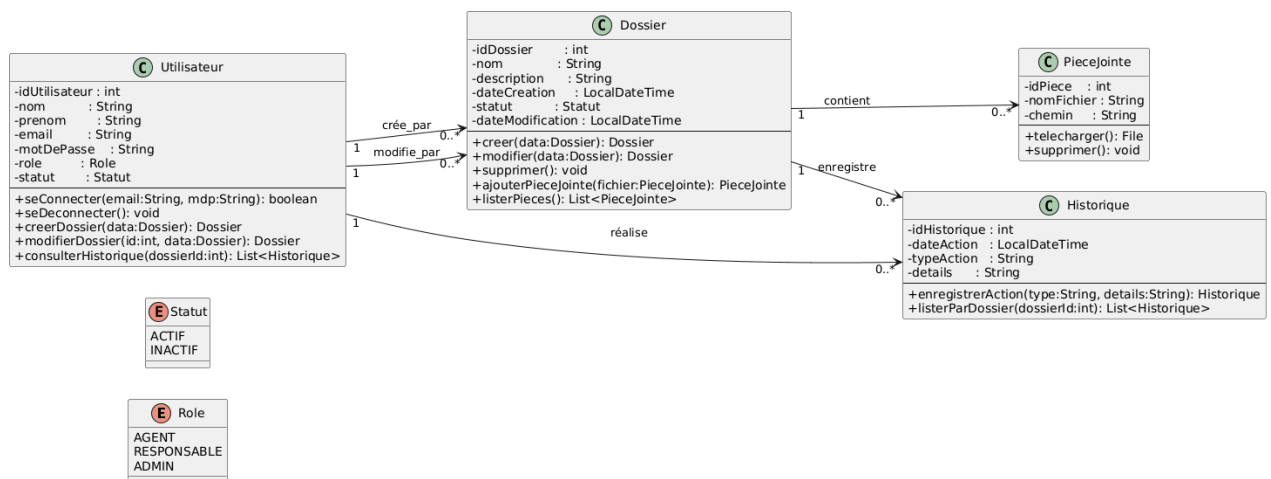
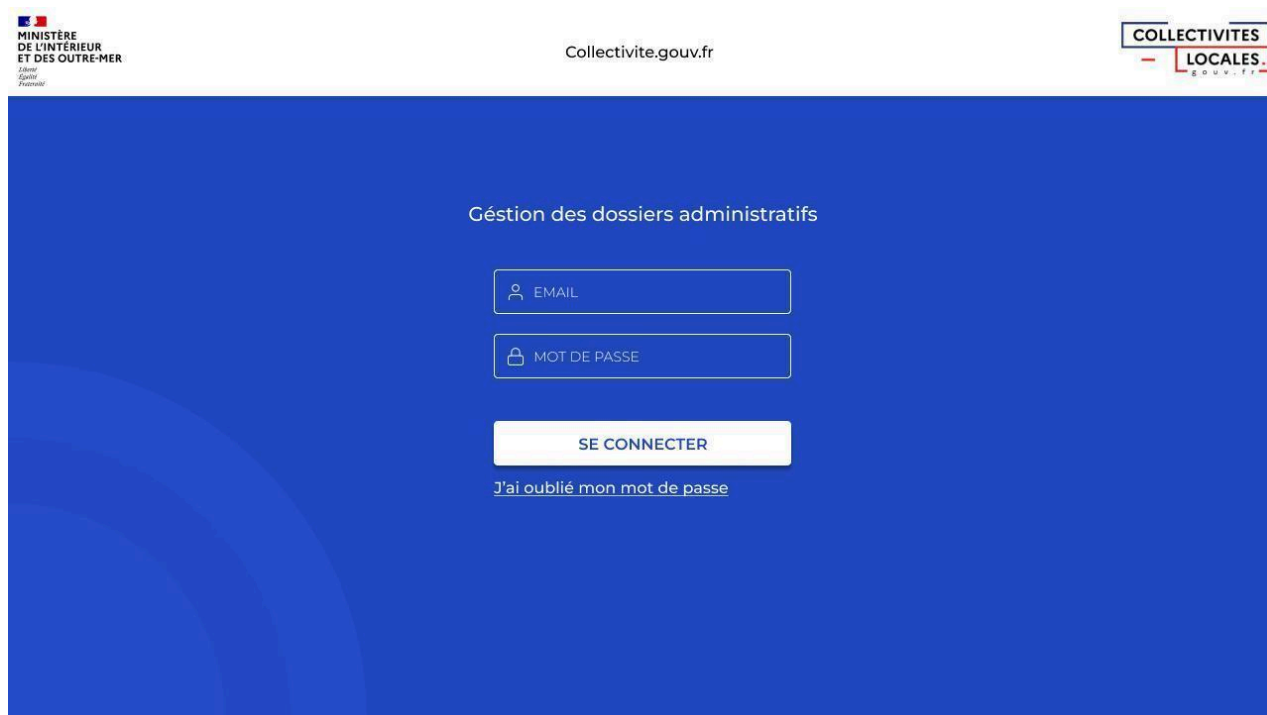


Figure 9 : Diagramme de classes

9. Maquettes filaires des écrans clés et parcours de navigation

9.1 Page de connexion



The screenshot shows the login page of Collectivite.gouv.fr. The page has a blue background with a subtle wave pattern on the left. At the top left is the logo of the Ministry of the Interior and Overseas. At the top center is the URL 'Collectivite.gouv.fr'. At the top right is the 'COLLECTIVITES LOCALES' logo. The main content area is titled 'Gestion des dossiers administratifs'. Below this title are three input fields: 'EMAIL' with a person icon, 'MOT DE PASSE' with a lock icon, and a 'SE CONNECTER' button. Below the button is a link that says 'J'ai oublié mon mot de passe'.

MINISTÈRE
DE L'INTÉRIEUR
ET DES OUTRE-MER
*Liberté
Égalité
Fraternité*

Collectivite.gouv.fr

COLLECTIVITES
— LOCALES —

Gestion des dossiers administratifs

EMAIL

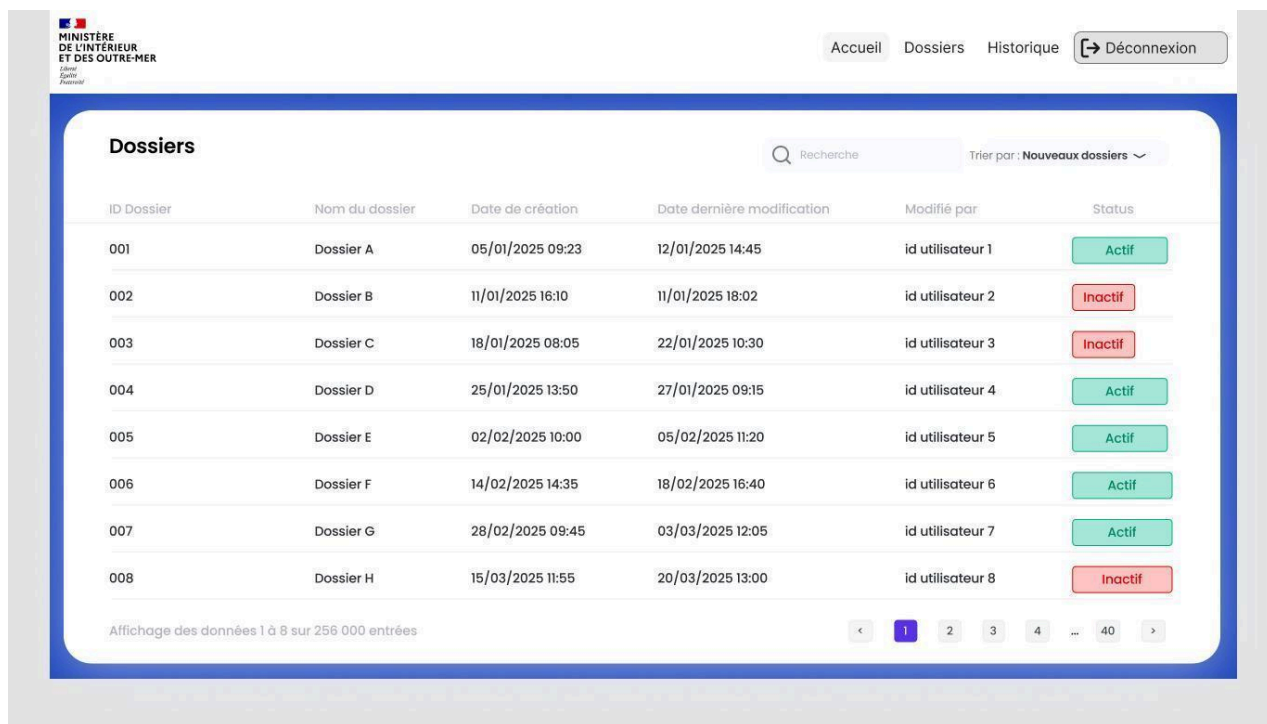
MOT DE PASSE

SE CONNECTER

[J'ai oublié mon mot de passe](#)

Figure 10 : Capture d'écran de la page de connexion

9.2 Tableau de bord (Accueil)



The screenshot shows a web application interface for the Ministry of the Interior and Overseas. At the top, there is a navigation bar with the logo of the Ministry, the text 'Collectivite.gouv.fr', and a 'Déconnexion' button. Below the navigation bar, the main content area is titled 'Dossiers'. It features a search bar and a dropdown menu for sorting. The main part of the interface is a table with 8 columns: ID Dossier, Nom du dossier, Date de création, Date dernière modification, Modifié par, and Status. The table contains 8 rows of data, each representing a dossier. The status of each dossier is indicated by a green 'Actif' button or a red 'Inactif' button. At the bottom of the table, there is a pagination bar showing 'Affichage des données 1 à 8 sur 256 000 entrées' and a set of navigation buttons.

ID Dossier	Nom du dossier	Date de création	Date dernière modification	Modifié par	Status
001	Dossier A	05/01/2025 09:23	12/01/2025 14:45	id utilisateur 1	Actif
002	Dossier B	11/01/2025 16:10	11/01/2025 18:02	id utilisateur 2	Inactif
003	Dossier C	18/01/2025 08:05	22/01/2025 10:30	id utilisateur 3	Inactif
004	Dossier D	25/01/2025 13:50	27/01/2025 09:15	id utilisateur 4	Actif
005	Dossier E	02/02/2025 10:00	05/02/2025 11:20	id utilisateur 5	Actif
006	Dossier F	14/02/2025 14:35	18/02/2025 16:40	id utilisateur 6	Actif
007	Dossier G	28/02/2025 09:45	03/03/2025 12:05	id utilisateur 7	Actif
008	Dossier H	15/03/2025 11:55	20/03/2025 13:00	id utilisateur 8	Inactif

Figure 11 : Capture d'écran du tableau de bord dossiers

9.3 Formulaire de création/modification de dossier



The screenshot shows a web application interface for the Ministry of the Interior and Overseas. At the top, there is a navigation bar with the logo of the Ministry, the text 'Collectivite.gouv.fr', and a 'Déconnexion' button. Below the navigation bar, the main content area is titled 'Création de dossier'. It features a form with the following fields: 'Nom du dossier' (text input), 'Description' (text area), 'Date' (dropdown menu), and 'Ajouter des pièces jointes' (button with an upload icon). At the bottom of the form, there are two buttons: 'Enregistrer' and 'Annuler'.

Figure 12 : Capture d'écran du formulaire de création/modification de dossier

10. Spécifications techniques détaillées

10.1 Sécurité des données

- **Chiffrement :**
 - Les mots de passe sont stockés en base de données sous forme de hachage fort (ex : bcrypt, Argon2).
 - Les données sensibles (pièces jointes, informations personnelles) sont chiffrées au repos (AES-256) et en transit (HTTPS/TLS).
- **Protection contre les injections :**
 - Utilisation systématique de requêtes préparées (paramétrées) pour toutes les interactions avec la base de données.
- **Validation côté serveur :**
 - Toutes les données reçues du client sont validées côté serveur (types, tailles, formats, listes blanches).

10.2 Authentification et gestion des sessions

- **Authentification forte :**
 - Mot de passe complexe requis, politique de renouvellement, verrouillage après X tentatives échouées.
 - Possibilité d'authentification à deux facteurs (2FA) pour les administrateurs.
- **Gestion sécurisée des sessions :**
 - Jetons d'authentification (JWT), expiration automatique, renouvellement sécurisé.
 - Protection contre le vol de session (SameSite cookies, HttpOnly).

10.3 Gestion des autorisations

- **Rôles et droits :**
 - Accès différencié selon le rôle (agent, responsable, admin).
 - Vérification systématique des droits à chaque action sensible (création, modification, suppression, gestion utilisateurs).

- **Journalisation :**

- Toutes les actions sensibles (modification, suppression, gestion des droits) sont tracées dans l'historique (audit log) avec horodatage et identification de l'utilisateur.

10.4 Gestion des exceptions et des erreurs

- **Gestion centralisée des exceptions :**

- Les erreurs applicatives sont capturées et traitées pour éviter les fuites d'informations sensibles.
- Les messages d'erreur affichés à l'utilisateur sont génériques, les détails techniques sont logués côté serveur.

- **Surveillance et alertes :**

- Mise en place de logs applicatifs et d'alertes en cas d'anomalie (tentatives d'accès non autorisé, erreurs critiques).

10.5 Conformité ANSSI et RGPD

- **Respect des recommandations ANSSI :**

- Mise à jour régulière des dépendances, désactivation des services inutiles, configuration sécurisée des serveurs.

- **RGPD :**

- Droit à l'oubli (suppression/masquage des données personnelles sur demande).
- Consentement explicite pour la collecte des données.
- Minimisation de la collecte des données (uniquement ce qui est nécessaire).

10.6 Accessibilité (RGAA)

- **Respect des critères RGAA niveau AA :**

- Navigation clavier, contrastes suffisants, textes alternatifs, structure des pages claire.
- Tests d'accessibilité automatisés et manuels à chaque itération.

10.7 Performance et éco-conception

- **Optimisation des requêtes :**
 - Indexation des champs de recherche, pagination des listes.
- **Sobriété des interfaces :**
 - Limitation des scripts et des images lourdes, chargement différé des ressources.
- **Surveillance de la consommation :**
 - Suivi des temps de réponse, alertes en cas de dépassement.

10.8 Déploiement et intégration continue

- **Conteneurisation :**
 - Utilisation de Docker pour isoler les services.
- **CI/CD :**
 - Pipelines automatisés pour les tests, la vérification des vulnérabilités, le déploiement.
- **Sauvegardes et restauration :**
 - Sauvegardes régulières de la base de données, procédures de restauration testées.

11. Points clés de la conformité sécuritaire

- **Veille régulière** sur les vulnérabilités (CVE, bulletins ANSSI).
- **Mise à jour** des dépendances et correctifs de sécurité.
- **Tests de pénétration** réguliers (outils automatisés + audits manuels).
- **Documentation** des incidents et des mesures correctives.

12. Plan de test

12.1 Fonctionnalité "Création de dossier"

ID	Scénario	Prérequis	Étapes	Résultat attendu	Résultat obtenu	Statut
T01	Création d'un dossier avec données valides	Utilisateur connecté avec droits suffisants	1. Accéder au tableau de bord 2. Cliquer sur "Créer un dossier" 3. Remplir tous les champs obligatoires 4. Cliquer sur "Enregistrer"	Dossier créé avec succès, message de confirmation affiché, redirection vers la liste des dossiers	Conforme	Succès
T02	Création d'un dossier avec données manquantes	Utilisateur connecté avec droits suffisants	1. Accéder au tableau de bord 2. Cliquer sur "Créer un dossier" 3. Laisser des champs obligatoires vides 4. Cliquer sur "Enregistrer"	Messages d'erreur affichés, formulaire non soumis	Conforme	Succès
T03	Création d'un dossier sans droits suffisants	Utilisateur avec droits limités	1. Accéder au tableau de bord 2. Tenter d'accéder à "Créer un dossier"	Accès refusé, message explicatif	Conforme	Succès

Tableau 9 : Plan de test de la fonctionnalité "Création de dossier"

12.2 Jeu d'essai "Création de dossiers"

Champ	Valeur de test	Validation
Nom du dossier	"Dossier Urbanisme 2025-042"	Valide
Description	"Demande de permis de construire pour extension"	Valide
Date	"02/05/2025"	Valide
Pièce jointe	Fichier PDF de 15 Mo	Invalide (taille max 10 Mo)

Tableau 10 : Jeu d'essai de la fonctionnalité "Création de dossier"

13. Extraits de code

13.1 Entité Dossier

```
import javax.persistence.*;

@Entity
public class Dossier {
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String description;
    private String statut;

    @ManyToOne
    @JoinColumn(name = "cree_par")
    private Utilisateur creePar;

    @ManyToOne
    @JoinColumn(name = "modifie_par")
    private Utilisateur modifiePar;

    // Getters, setters, constructeur vide et complet
}

```

Figure 13 : Extrait de code de l'entité Dossier

13.2 Entité Utilisateur

```

import javax.persistence.*;
@Entity
public class Utilisateur {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String prenom;
    private String email;
    private String motDePasse;
    private String role; // "agent", "responsable", "admin"
    private String statut; // "actif", "inactif"

    // Getters, setters, constructeur vide et complet
}

```

Figure 14 : Extrait de code de l'entité Utilisateur

13.3 Repository JPA

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface DossierRepository extends JpaRepository<Dossier,
Long> {
    // Méthodes CRUD de base
}

public interface UtilisateurRepository extends
JpaRepository<Utilisateur, Long> {
    Utilisateur findByEmail(String email);
}
```

Figure 15 : Extrait de code de la repository Dossier

13.4 Service métier (exemple création dossier)

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service
public class DossierService {
    @Autowired
    private DossierRepository dossierRepository;
    @Autowired
    private HistoriqueService historiqueService;

    public Dossier creerDossier(Dossier dossier, Utilisateur
utilisateur) {
        dossier.setCreePar(utilisateur);
        dossier.setStatut("actif");
        Dossier saved = dossierRepository.save(dossier);
        historiqueService.logAction(utilisateur, saved, "CREATION");
        return saved;
    }
}
```

```
}
```

Figure 16 : Extrait de code du service de création de dossier

13.5 Contrôleur REST sécurisé

```
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import java.util.List;

@RestController
@RequestMapping("/api/dossiers")
public class DossierController {

    @Autowired
    private DossierService dossierService;

    @GetMapping
    public List<Dossier> getAllDossiers() {
        return dossierService.getAllDossiers();
    }

    @PostMapping
    public Dossier createDossier(@RequestBody Dossier dossier,
    @RequestParam Long utilisateurId) {
        // Vérifier le rôle de l'utilisateur ici (sécurité)
        Utilisateur utilisateur =
utilisateurService.findById(utilisateurId);
        return dossierService.creerDossier(dossier, utilisateur);
    }
}
```

Figure 17 : Extrait de code du contrôleur REST

13.6 Sécurité (Spring Security basique)

xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Dans `application.properties`, on configure le mot de passe admin pour les tests :

```
spring.security.user.name=admin
spring.security.user.password=admin123
```

13.7 Configuration MySQL

Dans `application.properties` :

```
spring.datasource.url=jdbc:mysql://localhost:3306/gestion_dossiers
```

```
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Figure 18 : Configuration Spring Boot Mysql

14. Programmation Orientée Objet : Facilitation de la maintenance et de l'évolution

La Programmation Orientée Objet (POO) structure notre application autour de classes représentant les entités métier (Dossier, Utilisateur, Historique). Cette approche présente plusieurs avantages pour la maintenance et l'évolution du logiciel :

- Encapsulation : Les données et les comportements sont regroupés dans des classes, ce qui protège l'intégrité des données et limite les effets de bord.

Par exemple, toute modification d'un dossier passe par des méthodes dédiées qui appliquent les règles métier et les contrôles de sécurité.

- Modularité : L'architecture en couches (contrôleurs, services, repositories, entités) permet de modifier une partie de l'application (ex. la couche d'accès aux données) sans impacter les autres, ce qui facilite l'ajout de nouvelles fonctionnalités ou la correction de bugs.
- Héritage et polymorphisme : La gestion des rôles utilisateurs (Agent, Responsable, Admin) peut être étendue facilement grâce à l'héritage et au polymorphisme, permettant d'ajouter de nouveaux rôles ou droits sans réécrire l'ensemble du code.
- Réutilisabilité : Les services transverses (authentification, journalisation, validation) sont factorisés et réutilisés dans toute l'application, ce qui réduit la duplication de code et accélère l'évolution du projet.

Grâce à la POO, notre application reste évolutive, robuste et facilement maintenable, même en cas d'évolution des besoins métiers ou des exigences réglementaires (RGPD, ANSSI).

15. Éléments de sécurité mis en œuvre dans l'application

La sécurité est une priorité dans la conception et le développement de notre application. Les principales mesures mises en œuvre sont :

15.1 Sécurisation des données

- Hachage des mots de passe avec BCrypt pour empêcher le vol de mots de passe en cas de fuite de la base.
- Chiffrement des données sensibles au repos (AES-256) et en transit (HTTPS/TLS).
- Validation systématique des entrées côté serveur pour éviter les injections SQL et les attaques XSS.

15.2 Authentification et contrôle d'accès

- Authentification forte avec politique de mots de passe complexes et expiration régulière.
- Gestion sécurisée des sessions (cookies HttpOnly, SameSite, expiration automatique).
- Contrôle d'accès basé sur les rôles (RBAC) : chaque action (création, modification, suppression, gestion utilisateurs) est protégée par des vérifications de droits via les annotations Spring Security ([@PreAuthorize](#)).

15.3 Journalisation et traçabilité

- Audit log : toutes les actions sensibles (création, modification, suppression) sont tracées avec l'identifiant de l'utilisateur et l'horodatage.
- Surveillance et alertes : des alertes sont générées en cas de comportement suspect ou d'accès non autorisé.

15.4 Protection contre les attaques courantes

- Protection CSRF activée pour les formulaires sensibles.
- En-têtes de sécurité HTTP (Content-Security-Policy, X-Content-Type-Options, etc.).
- Rate limiting pour limiter les tentatives de connexion et prévenir les attaques par force brute.

15.5 Conformité réglementaire

- Respect du RGPD : droit à l'oubli, minimisation des données, consentement explicite.
- Conformité ANSSI : mise à jour régulière des dépendances, configuration sécurisée des serveurs, documentation des incidents.

Ces éléments sont intégrés à tous les niveaux de l'application, de la conception à la mise en production, et font l'objet d'une veille sécuritaire continue, comme décrit dans la section précédente.

ANNEXES

Tableau 1 : Domaines d'activité principaux de SecureApp	1
Figure 1 : Organigramme de l'entreprise SecureApp	2
Tableau 2 : Equipes du service Conception et Développement	2
Tableau 3 : Phases du projet et planning	7
Figure 2 : Diagramme de Gantt du projet	7
Tableau 4 : Acteurs et rôles	8
Figure 4 : MCD (Modèle Conceptuel de Données) du projet	10
Figure 5 : MLD (Modèle Logique de Données) du projet	11
Tableau 5 : Table SQL Utilisateur	12
Tableau 6 : Table SQL Dossier	12
Tableau 7 : Table SQL Historique	13
Tableau 8 : Table SQL Pièces jointes	13
Figure 6 : Script SQL de création de la BDD	15
Figure 7 : Diagramme de cas d'utilisation	16
Figure 8 : Diagramme de séquences	17
Figure 9 : Diagramme de classes	17
Figure 10 : Capture d'écran de la page de connexion	18
Figure 11 : Capture d'écran du tableau de bord dossiers	19
Figure 12 : Capture d'écran du formulaire de création/modification de dossier	19
Tableau 9 : Plan de test de la fonctionnalité "Création de dossier"	23
Tableau 10 : Jeu d'essai de la fonctionnalité "Création de dossier"	23
Figure 13 : Extrait de code de l'entité Dossier	24
Figure 14 : Extrait de code de l'entité Utilisateur	25
Figure 15 : Extrait de code de la repository Dossier	25
Figure 16 : Extrait de code du service de création de dossier	26
Figure 17 : Extrait de code du contrôleur REST	26
Figure 18 : Configuration Spring Boot Mysql	27